Scaled Dot-Product Attention · a study excerpt

Classroom excerpt adapted for Paper2Explorer (based on Vaswani et al., 2017)

1 Introduction

Sequence models compute a representation of every token by looking at other tokens. An attention function maps a query and a set of key-value pairs to an output, where the query, keys, values and output are all vectors. The output is a weighted sum of the values; the weight of each value comes from a compatibility function of the query with the corresponding key.

2 Method

2.1 Scaled Dot-Product Attention

The input consists of queries and keys of dimension d_k and values of dimension d_v. We compute the dot products of the query with all keys, divide each by sqrt(d_k), and apply a softmax function to obtain the weights on the values. In practice the queries, keys and values are packed into matrices Q, K and V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q K^{T} / \sqrt{d_k} \right) V \qquad (1)$$

For large d_k the dot products grow large in magnitude, pushing the softmax into regions with extremely small gradients. Scaling by 1/sqrt(d_k) counteracts this.

2.2 Step-by-step procedure

Step 1. Compute the score matrix S = Q K^T (one score per query-key pair).
Step 2. Scale the scores: S' = S / sqrt(d_k).
Step 3. Normalise each row with softmax so the weights of one query sum to 1.
Step 4. Multiply the weights by V to obtain the output for each query.

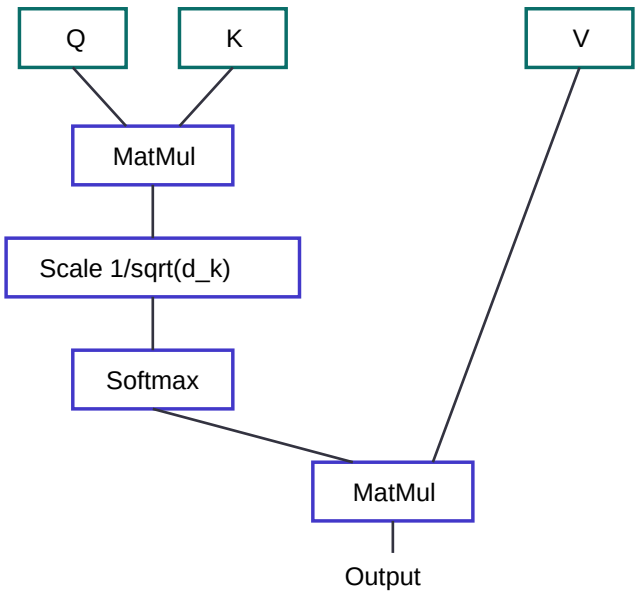

Figure 1: Scaled Dot-Product Attention. Q and K are multiplied, scaled by 1/sqrt(d_k), normalised with softmax, and the resulting weights are applied to V.

2.3 Multi-Head Attention
Instead of a single attention function with d_model-dimensional keys, values and queries, we linearly project them h times with different learned projections:

MultiHead(Q, K, V) = Concat(head_1, ..., head_h) W^O (2)
where head_i = Attention(Q W_i^Q, K W_i^K, V W_i^V)

We use h = 8 heads with d_k = d_v = d_model / h = 64.

3 Analysis
Table 1 compares layer types by computational complexity per layer, where n is the sequence length and d the representation dimension.

Table 1: Complexity per layer and number of sequential operations.

Layer type	Complexity per layer	Sequential operations
Self-attention	O(n^2 * d)	O(1)
Recurrent	O(n * d^2)	O(n)
Convolutional	O(k * n * d^2)	O(1)

4 Worked example (for students)
Take three tokens with d_k = 2: Q = K = [[1, 0], [0, 1], [1, 1]] and V = [[1, 0], [0, 1], [0.5, 0.5]]. The score of token 3 with itself is 2, which after scaling by 1/sqrt(2) becomes about 1.41. A larger score gives a larger weight after softmax, so each output mixes the values it attends to most.

5 Limitations of this excerpt
This excerpt explains one component only. It does not reproduce training, datasets or experimental results of the original paper.